How LLMs Process Information

A Geometric View Inside Transformer Computation

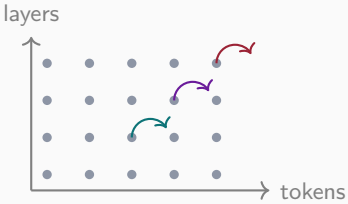
Agus Sudjianto

Based on *Learning Geometric Algebra Through Transformer Geometry*

https://github.com/asudjianto-xml/layer-time-geometry

Roadmap

1. **The Standard Picture** 10 min
   Tokens, layers, hidden states
2. **Layers as Rotations** 12 min
   Versors, bivectors, rotation planes
3. **When Order Matters** 10 min
   Commutators, curvature, capacity
4. **The Decode Frontier** 12 min
   Generation, hallucination signatures
5. **Seeing Inside & Steering** 10 min
   Dependency, intervention, the big picture
6. **Q&A** 6 min



The layer–token grid

The Standard Picture

What happens when you prompt an LLM?

- Each token becomes a **hidden-state vector** $H^{(l,t)} \in \mathbb{R}^{3584}$
- $L = 28$ layers, T tokens $\Rightarrow$ a grid of $(28 \times T)$ vectors
- Standard view: "matrix multiplies and attention." True but uninformative.

What is each layer actually doing to these vectors?

The layer–token grid

Every forward pass produces a **grid** of hidden states:

$$H : (l, t) \longmapsto \mathbb{R}^k$$

- Vertical axis: **layers** (depth)
- Horizontal axis: **token positions** (context)
- Every dot is a high-dimensional vector

We need a language that describes not just *how much* each layer changes the vectors, but *in which directions*.

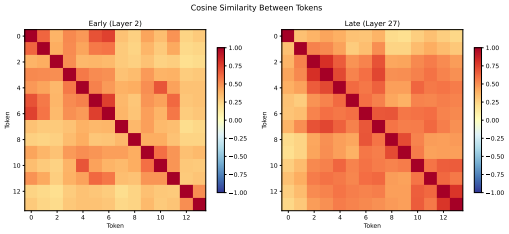


Figure: Cosine similarity between hidden states at adjacent layers. Values near 1 = small change; dips = layers that substantially redirect the vector. Each layer reshapes meaning by a different amount.

Layers as Rotations

Building an honest coordinate system

Raw hidden states live in $\mathbb{R}^{3584}$ with a **distorted** metric: distances and angles are unreliable.

Whitening (PCA + rescaling):

$$\mathbb{R}^{3584} \xrightarrow{\text{PCA}} \mathbb{R}^{256} \xrightarrow{\text{rescale}} \text{Covariance} = I_{256}$$

- Retains $> 99\%$ of variance
- Establishes an **orthonormal frame**
- Angles and rotations become *meaningful*
- Required for valid GA in $\mathrm{Cl}(256, 0)$

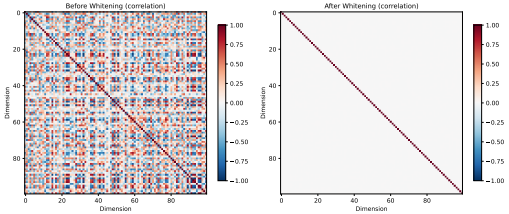


Figure: Eigenvalue spectrum of the covariance. After whitening, retained dimensions have equal variance — distances and angles become trustworthy.

What a layer does: rotation × stretch

Each layer transition decomposes as:

$$\boxed{T^{(l)} = \underbrace{R^{(l)}}_{\text{rotation}} \cdot \underbrace{P^{(l)}}_{\text{stretch}}}$$

Rotation (grade-2, rotor)

- Reorients the hidden state — moves meaning to *different directions*
- Preserves vector lengths
- Generator: bivector $B^{(l)} = \log(R^{(l)})$

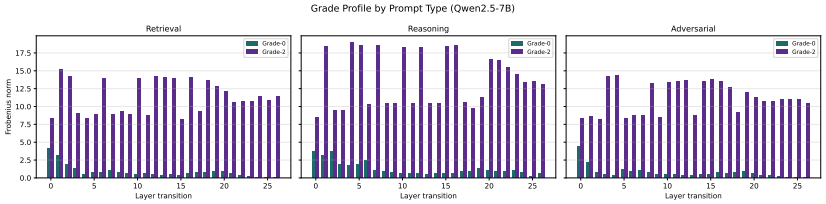
Stretch (grade-0, metric factor)

- Amplifies or suppresses specific components — *adjusts emphasis*
- Preserves directions
- Near-identity in practice

Key finding: Rotation dominates stretch by **10–15**× in norm.

Layers mostly rotate — they redirect meaning, not rescale it.

Rotation dominates: the grade profile



Reading the figure: Each panel shows one prompt type. Blue bars = rotation (grade-2) norm; purple bars = stretch (grade-0) norm, per layer. **Takeaway:** Blue towers over purple at every layer — rotation outweighs stretch by **10–15**×. Layers reshape *which directions* matter, not how big they are.

Bivectors: the plane of rotation

A rotation in $\mathbb{R}^{256}$ doesn't have an "axis." It has a **plane**.

The bivector $B^{(l)} = \log(R^{(l)})$ is a **skew-symmetric matrix** encoding:

- **Which plane(s)** the layer rotates in
- **How much** (the angle θ)

Principal plane decomposition: $B^{(l)} = \sum_{j=1}^{128} \sigma_j \, \hat{B}_j$.

Typically **3–5 dominant planes** per layer.

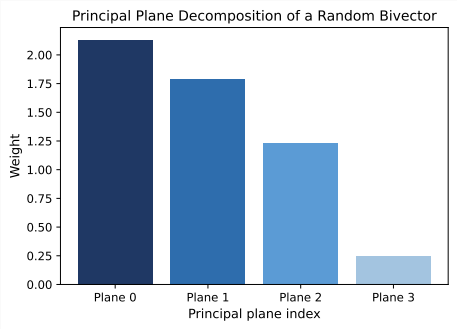


Figure: Singular values of the bivector. The first few bars dominate — rotation is concentrated in 3–5 planes, not all 128.

A layer is a field of rotations

Attention makes the transformation **position-dependent**:

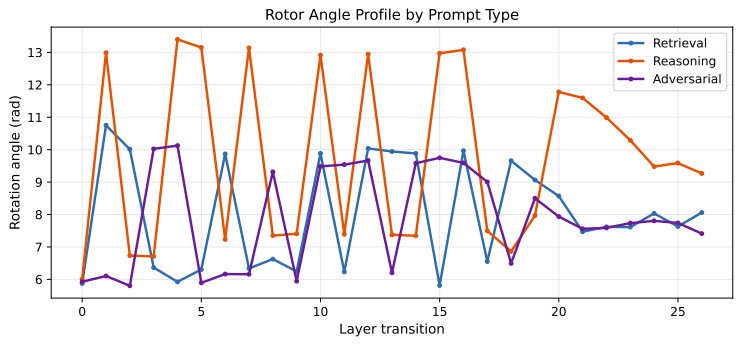
$$H^{(l+1)}(t) \;=\; R^{(l)}(t)\, P^{(l)}(t)\;\; H^{(l)}(t)$$

Each token position gets its **own rotor**.

Different rotation at each position

A layer is not "a rotation" — it is a **field of rotors**, each rotating the hidden state in its own plane by its own angle.

Rotor angles across layers



Reading the figure: Each curve traces rotation angle $\tau^{(l)}$ (degrees) per layer for one prompt type. Higher = bigger rotation. **Takeaway:** Early layers rotate aggressively (40–60°); deeper layers make finer adjustments. Different tasks produce *different* profiles — the model adapts its rotation schedule to the input.

When Order Matters

Do layers commute?

The **commutator** asks: does it matter which order I apply two layers' rotations?

$$[B^{(l)},\ B^{(l+1)}] \ = \ B^{(l)}B^{(l+1)} - B^{(l+1)}B^{(l)}$$

- $= 0$: layers are **independent** — swapping them wouldn't change the result
- $\neq 0$: layer $l{+}1$ **builds on** layer l's rotation — sequential processing matters
- Non-zero commutator = *genuinely sequential* reasoning, not parallel operations

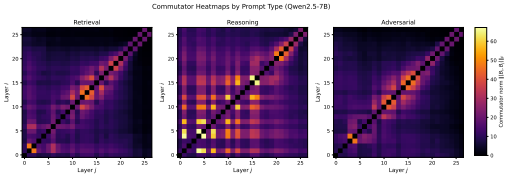


Figure: Each bar = $\|[B^{(l)}, B^{(l+1)}]\|_F$ for one layer pair. Tall bars = strong sequential coupling.

Capacity: the model's non-commutative budget

Capacity = total sequential computation. Sum commutator norms over all adjacent layer pairs: $C_{\text{acc}} = \sum_{\text{adj. pairs}} \|[B^{(l)}, B^{(l+1)}]\|_F$

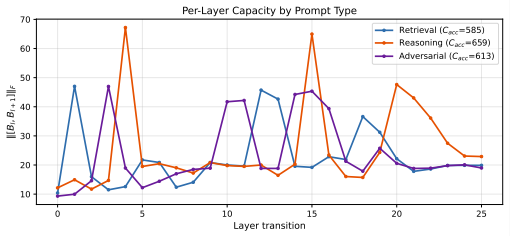


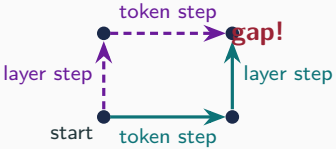
Figure: Cumulative capacity C_{acc} vs. layer. Steeper climb = more sequential computation invested up to that depth.

Prompt	C_{acc}	**Peak**
Retrieval	584.9	Layer 1
Reasoning	658.5	Layer 4
Adversarial	613.4	Layer 3

- Reasoning demands **13% more** capacity — more sequential steps needed
- Peak layer shifts with task: the model allocates its "budget" **adaptively**

Walking in circles: holonomy and curvature

Transport a vector around a closed loop on the grid. If it comes back *changed*, the space is curved.



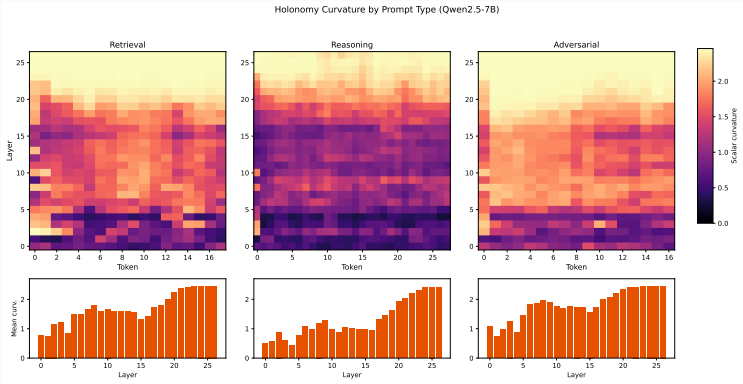
Compose all four rotors around the cell — the **loop rotor**:

$$R_{\text{loop}} = R_{\uparrow} R_{\rightarrow} R_{\uparrow}^{-1} R_{\rightarrow}^{-1}$$

If $R_{\text{loop}} \neq I$: the grid is **curved**. $\Omega = \log(R_{\text{loop}})$ tells you *which plane*.

Intuition: how a layer transforms a token *depends on which other tokens it has seen* — layer and context are entangled.

Curvature across the grid

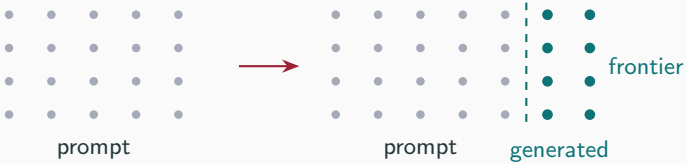


Reading the figure: Each heatmap cell is one plaquette (l, t). Color intensity = holonomy deficit $\|R_{\text{loop}} - I\|_F$. Brighter = more curvature. **Takeaway:** Non-zero curvature means layer and token processing are **entangled**. Reasoning prompts show the strongest curvature.

The Decode Frontier

From static analysis to generation

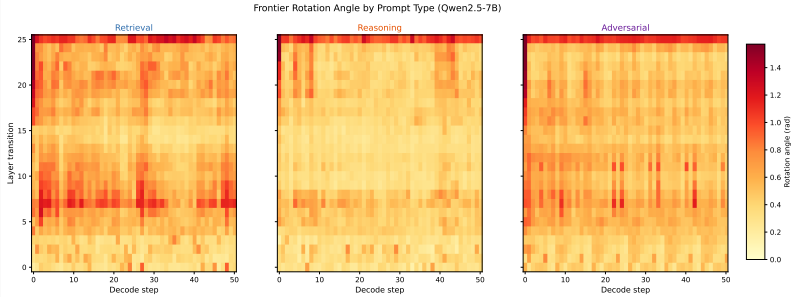
Everything so far: one prompt, one forward pass. But LLMs **generate** — one token at a time.



Each new token adds a **column** to the grid. The **decode frontier** is the evolving right edge.

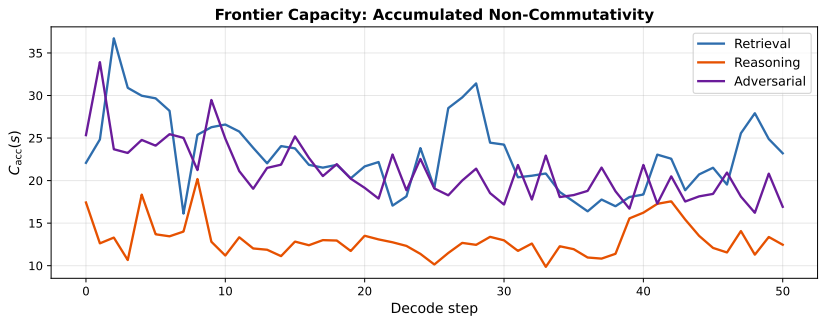
Question: How does the geometry change as the model generates?

Rotation planes drift during generation



Reading the figure: Each curve tracks the rotation angle at the newest generated token, plotted against decode step. Each line is a different layer. **Takeaway:** Rotation planes are **not preserved** across steps — they drift as context grows. Adversarial prompts show periodic cycling that mirrors stuck token loops.

Frontier capacity decays during generation



Reading the figure: x-axis = decode step (tokens generated so far); y-axis = frontier capacity C_{acc} at that step. Each curve is one prompt type. **Takeaway:** Capacity decays as the model generates — its sequential-reasoning budget *drains*. Retrieval stays high and stable; adversarial declines into periodic cycling, signaling the model is running low on compositional resources.

Hallucination as geometric collapse

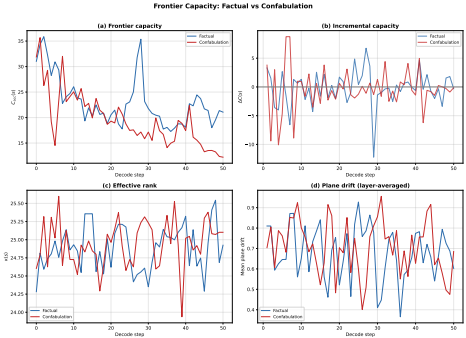


Figure: Two capacity-vs-step curves: factual (blue) vs. confabulated (red). Steeper slope = faster loss of sequential reasoning budget.

Comparing factual vs. confabulated generation:

	Factual	**Confab.**
Capacity decay	-0.18	-0.28
Curvature accel.	0.186	0.221

- **56% faster** capacity decay
- Curvature not just higher but **accelerating**
- Model loses geometric structure as it fabricates

Hallucination has a measurable geometric signature *during* generation.

Seeing Inside and Steering

Which layers matter?

Dependency asks: if I perturbed this layer's output, how much would the prediction change? Measured via gradient norm: $D_l = \|\partial s / \partial H^{(l)}\|_F$.

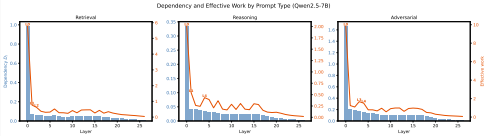


Figure: Each bar = gradient-based dependency D_l per layer. Taller = the output is more sensitive to that layer's contribution.

Effective work combines "how much the layer rotates" with "how much the output cares": $W_l = D_l \cdot \theta^{(l)}$

- The **loudest** layer (biggest rotation) is often *not* the most important
- The small stretches (grade-0) sit at high-dependency layers — geometrically quiet but **functionally decisive**

Effective work pinpoints where the model is both *active* and *consequential*.

Cayley steering: from detection to control

The Cayley bivector between *query-only* and *context-augmented* hidden states tells us **where** and **in which plane** context acts:

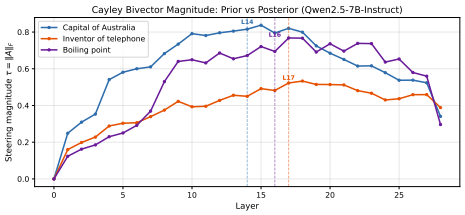


Figure: $\tau^{(l)}$ per layer for three counterfactual prompts. The peak identifies where context most strongly rotates the hidden state.

Prompt	Layer	τ	Output
Capital of Australia	14	0.84	Sydney
Inventor of telephone	17	0.53	Tesla
Boiling point	16	0.77	250°C

The **steering recipe**:

1. Compute $\tau^{(l)}$ — find the peak layer
2. Extract the principal plane (v_1, v_2) of the Cayley bivector
3. Apply a Rodrigues rotation by angle α in that plane during generation

One GA object — the Cayley bivector — provides detection, diagnosis, and control.

When context fails: steering past parametric resistance

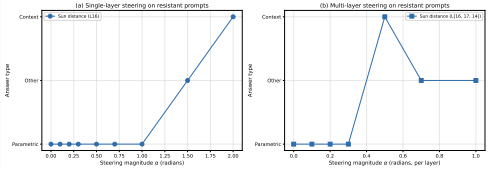


Figure: Steering sweep on a resistant prompt. (a) Single-layer: requires α=2.0. (b) Multi-layer: context answer at α=0.5 with a narrow effective window.

Context says "Earth–Sun distance = **50 million miles**." Model ignores it, answers "93 million miles."

The Cayley bivector *detects* context influence ($\tau = 0.70$ at layer 16) — the hidden states are rotated, but not enough to flip the output.

Steering	α	**Result**
None	—	93 million (parametric)
Single L16	2.0	1 million (context)
Multi L16,17,14	**0.5**	**50 million (exact)**

Takeaway: Multi-layer Cayley steering at moderate α overrides parametric resistance that context alone cannot — a geometrically targeted nudge in the model's own rotation planes.

The Big Picture

A transformer is a geometric field system

The entire framework is captured in **three fields** over a discrete grid:

$$
\begin{aligned}
H : \mathcal{G} &\longrightarrow \mathbb{R}^k && \text{(hidden-state field)} \\
R : \mathcal{G} &\longrightarrow \mathrm{Spin}(k) && \text{(rotor field)} \\
\Omega : \mathcal{G} &\longrightarrow \mathfrak{so}(k) && \text{(curvature field)}
\end{aligned}
$$

$$\textit{Vectors} \longrightarrow \textit{Rotations} \longrightarrow \textit{Curvature}$$

What GA reveals that norms cannot

Scalar summaries tell you **how much**. GA tells you **how much** and **in which direction**.

Static (single forward pass):

1. Which planes each layer rotates in
2. Which planes the curvature acts in
3. Which planes layers fail to commute in
4. Which planes the output depends on

Dynamic (during generation):

5. How rotation planes drift or collapse
6. Whether capacity decay signals hallucination
7. Whether reasoning is causally aligned with context

A transformer is a discrete geometric field system whose computation is governed by rotations and curvature.

Summary

Concept	Intuition	GA object
Versor decomp.	How much a layer rotates vs. stretches	$T = R \cdot P$
Bivector field	Which directions each layer reshuffles	$B^{(l)} \in \mathfrak{so}(k)$
Commutator	Does layer order matter?	$[B^{(l)}, B^{(l+1)}]$
Capacity	Total sequential computation budget	$C_{\text{acc}} = \sum \|[\cdot]\|_F$
Holonomy	Are layers and context entangled?	R_{loop}
Dependency	Which layers the output cares about	$D_l = \|\partial s / \partial H^{(l)}\|_F$
Frontier	How geometry evolves during generation	Bivectors at decode edge

Key takeaways:

- Transformer layers are predominantly **rotations** in high-dimensional space
- **Non-commutativity** between layers is the geometric basis of sequential reasoning
- **Curvature** reveals where layer and token processing are entangled
- Hallucination produces a measurable **geometric signature** during generation
- Geometric understanding enables **plane-specific intervention**

Questions?

How LLMs Process Information: A Geometric View

Agus Sudjianto

Based on *Learning Geometric Algebra Through Transformer Geometry*

`https://github.com/asudjianto-xml/layer-time-geometry`

Model: Qwen2.5-7B (28 layers, whitened to $k = 256$) · Library: `layer_time_ga`